

Comprehensive Web Application Bug Bounty Testing Checklist & Methodology Summary

This document provides a deep, systematic approach to web application penetration testing and bug bounty hunting. It covers reconnaissance, vulnerability assessment, exploitation, and reporting across all major OWASP categories and common attack vectors.

Phase 1: Reconnaissance & Information Gathering

1.1 Passive Reconnaissance

Domain & Network Enumeration

- ☐ Identify all subdomains using:
 - subfinder, assetfinder, amass
 - DNS brute-forcing: dnsx, dnsenum
 - CNAME enumeration and alias discovery
 - Reverse DNS lookups
- ☐ Check WHOIS records for organizational info
- ☐ Identify IP ranges and ASN information
- ☐ Look for forgotten/deprecated subdomains on old hosting
- ☐ Search for archived versions on Wayback Machine
- ☐ Monitor GitHub for exposed credentials and API endpoints

Technology Stack Identification

- ☐ Use Wappalizer, BuiltWith, Shodan
- ☐ Identify web framework, CMS, server software
- ☐ Check HTTP headers for version information
- ☐ Enumerate SSL/TLS certificates (crt.sh, SSL Labs)
- ☐ Identify third-party services and APIs
- ☐ Check robots.txt, sitemap.xml, .well-known/ directories

URL & Endpoint Discovery

- ☐ Use waybackurls to find historical endpoints
- ☐ Crawl with hakrawler, gau (Google, Archive, URLScan)
- ☐ Check for API documentation (Swagger, OpenAPI, GraphQL)
- ☐ Brute-force common paths and extensions
- ☐ Identify admin panels, backup files, config files
- ☐ Look for .git, .svn, .env, .DS_Store exposed

1.2 Active Reconnaissance

Network Scanning

- ☐ Port scanning: nmap, masscan
- ☐ Service enumeration and version detection
- ☐ Check for open management interfaces (RDP, SSH, etc.)
- ☐ Identify cloud storage buckets (S3, GCS, Azure Blob)

Web Server Enumeration

- ☐ Fingerprint web server and version
- ☐ Test for HTTP methods (GET, POST, PUT, DELETE, TRACE, OPTIONS)
- ☐ Identify allowed HTTP methods per endpoint
- ☐ Check for TRACE method enabled (HTTP Response Splitting)
- ☐ Enumerate virtual hosts and hosted applications

API Enumeration

- ☐ Identify all API endpoints
- ☐ Document API authentication mechanisms
- ☐ Check for API versioning (v1, v2, etc.)
- ☐ Test for rate limiting and throttling
- ☐ Look for API keys in responses and headers

Phase 2: Authentication & Session Management

2.1 Authentication Testing

Credential Handling

- ☐ Test for default credentials (admin/admin, test/test, etc.)
- ☐ Check password strength requirements
- ☐ Test for SQL injection in login forms
- ☐ Test for NoSQL injection in login forms
- ☐ Look for username enumeration vulnerabilities
- ☐ Check for response timing differences (timing attacks)
- ☐ Test for brute-force protection and account lockout
- ☐ Verify rate limiting on authentication endpoints

Registration & Account Creation

- ☐ Test for account registration bypass
- ☐ Check email verification mechanisms
- ☐ Look for mass assignment/parameter pollution
- ☐ Test for weak password reset tokens
- ☐ Verify CAPTCHA bypass possibilities
- ☐ Check for privilege escalation via registration

OAuth & Third-Party Auth

- ☐ Verify OAuth token validation ☐ Check
- redirect URI validation (open redirect) ☐ Test
- for code/token replay attacks ☐ Verify scope
- limitations are enforced ☐ Check for state
- parameter validation ☐ Test for cross-site
- request forgery in auth ow

2.2 Session Management

Session Token Analysis

- ☐ Analyze session token entropy and randomness
- ☐ Check token storage mechanism (cookies, localStorage, sessionStorage)
- ☐ Verify secure and httpOnly flags on cookies
- ☐ Check SameSite cookie attribute
- ☐ Test for session fixation vulnerabilities
- ☐ Look for session token prediction/brute-force
- ☐ Check token expiration and timeout behavior
- ☐ Test for concurrent session limits

Session Handling

- ☐ Verify logout functionality clears all sessions
- ☐ Test for session replay attacks
- ☐ Check for cross-domain session sharing
- ☐ Verify privilege level enforced per session
- ☐ Test for session validation on sensitive operations

Phase 3: Authorization & Access Control

3.1 Broken Access Control (IDOR)

Horizontal Privilege Escalation

- ☐ Identify all numeric/sequential IDs (user ID, order ID, etc.)
- ☐ Test parameter manipulation:
 - Direct value change: ?id=123 → ?id=124
 - Base64 encoding: ?id=MTIz → modify to other values
 - Hash-based IDs: test predictability
 - UUIDs: test UUID enumeration
- ☐ Check all HTTP methods (GET, POST, PUT, PATCH, DELETE)
- ☐ Test for ID prediction in URLs and parameters
- ☐ Verify access control on API endpoints
- ☐ Check GraphQL for enumeration of other users' data

Vertical Privilege Escalation

- ☐ Test for role/privilege parameter tampering
- ☐ Check for direct admin panel access
- ☐ Verify privilege checks on sensitive operations
- ☐ Test for hidden parameters (role=admin, is_admin=true)
- ☐ Check for parameter pollution
- ☐ Test for privilege escalation via file upload/manipulation
- ☐ Look for JWT/token manipulation to elevate privileges

Endpoint Authorization

- ☐ Map all endpoints requiring authentication
 - ☐ Test unauthenticated access to protected resources
 - ☐ Verify proper authorization checks on all endpoints
 - ☐ Test for bypass via path manipulation:
 - /admin/users → /admin/./users
 - /admin/users → /admin/users/..
 - /admin/users → /ADMIN/users
 - /admin/users → /admin/users;.jpg
 - ☐ Check for case sensitivity bypass
 - ☐ Test for null byte injection (older systems)
-

Phase 4: Input Validation & Injection Attacks

4.1 SQL Injection

Detection

- ☐ Test all input fields for SQL injection:
 - Form inputs (login, search, filter)
 - URL parameters
 - HTTP headers (User-Agent, Referer, Cookie)
 - API parameters
- ☐ Use payloads: ', ", --, /**/, xp_, exec
- ☐ Check error messages for SQL syntax errors
- ☐ Test for time-based blind SQLi: SLEEP(5), WAITFOR
- ☐ Look for boolean-based blind SQLi
- ☐ Test for UNION-based SQLi

Exploitation

- ☐ Extract database version and structure
- ☐ Enumerate database users and privileges
- ☐ Dump sensitive data (users, credentials, PII)
- ☐ Check for second-order SQL injection
- ☐ Test for SQL injection in different contexts:
 - WHERE clauses
 - ORDER BY clauses
 - INSERT statements
 - UPDATE statements
- ☐ Verify protection mechanisms (WAF bypass)

Advanced SQL Injection

- [] Out-of-band SQL injection (DNS ex ltration)
- [] Stacked queries for arbitrary command execution
- [] Test for OS command execution via SQL:
 - xp_cmdshell (MSSQL)
 - sys_exec (MySQL)
 - COPY FROM PROGRAM (PostgreSQL)

4.2 NoSQL Injection

Detection & Exploitation

- [] Test JSON/MongoDB injection: {"\$ne": null}, {"\$gt": ""}
- [] Check for operator injection in API parameters
- [] Test for JavaScript injection in MongoDB
- [] Look for where clause injection
- [] Verify protection mechanisms

4.3 Command Injection

OS Command Injection

- [] Identify functions that execute system commands
- [] Test input parameters for injection:
 - Command separators: ;, |, ||, &, &&
 - Command substitution: `command`, \$(command)
 - Newline injection: %0a, %0d
- [] Test blind command injection with time delays
- [] Attempt out-of-band ex ltration (DNS, HTTP callbacks)
- [] Check for lter bypasses:
 - Case variation: PING vs ping
 - Encoding: URL encoding, unicode
 - Null bytes: ping%00.exe
 - Variable expansion: \${IFS} (bash)

4.4 Template Injection

Server-Side Template Injection (SSTI)

- [] Identify template engines (Jinja2, Freemarker, Velocity, etc.)
- [] Test for injection: {{7*7}}, \${7*7}, <%= 7*7 %>
- [] Exploit for RCE:
 - Python: {{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
 - Java: <#assign ex="freemarker.template.utility.Execute"?new()> \${ ex("whoami") }
 - PHP: {php}echo shell_exec('id');{/php}

Client-Side Template Injection

- [] Test Angular, Vue, React templates for injection
- [] Exploit prototype pollution in JavaScript
- [] Look for DOM-based template injection

4.5 XXE (XML External Entity)

Detection & Exploitation

- [] Identify XML processing endpoints
- [] Test with XXE payload:]>&x x e;
- [] Test for `le read` (local `le` disclosure)
- [] Attempt blind XXE with out-of-band ex ltration
- [] Test for SSRF via XXE
- [] Check for XXE in SVG uploads
- [] Look for XXE in PDF generation

4.6 LDAP & Directory Injection

- [] Test LDAP lters for injection
- [] Bypass authentication with wildcard: *
- [] Extract directory information
- [] Enumerate users and groups

Phase 5: Cross-Site Scripting (XSS)

5.1 Re ected XSS

Detection

- [] Test all user input elds for XSS:
 - URL parameters
 - Form inputs
 - API responses
 - HTTP headers (Referer, User-Agent)
- [] Basic payloads: `<script>alert('XSS')</script>`, `<img src=x onerror=alert('XSS')>`
- [] Test encoding bypass:
 - HTML encoding: `<script>`
 - URL encoding: `%3Cscript%3E`
 - Unicode: `\u003Cscript\u003E`
 - Hex: `<script>`

Filter Bypass

- [] Case variation: `<ScRiPt>`, `<SCRIPT>`
- [] Attribute obfuscation: `<img src=x on error=alert(1)>`
- [] Event handler bypass:
 - `onload`, `onerror`, `onmouseover`, `onkeypress`
 - `onfocus`, `onblur`, `onchange`
- [] Protocol handlers: `javascript:`, `data:`, `vbscript:`
- [] SVG vectors: `<svg/onload=alert(1)>`

- [] HTML5 vectors: `<video src=x onerror=alert(1)>`

Polyglot & Advanced XSS

- [] Context-aware payloads (HTML, JavaScript, URL, CSS contexts)
- [] Dangling markup injection
- [] CSS injection for exfiltration
- [] PostMessage XSS vulnerabilities

5.2 Stored XSS

Discovery

- [] Identify all user-controlled data storage:
 - Profiles (bio, avatar, etc.)
 - Comments, posts, reviews
 - Chat messages, notifications
 - File uploads with preview
- [] Test payload persistence
- [] Verify when and where stored XSS is reflected
- [] Check for second-order XSS

Exploitation

- [] Create payload for session hijacking (steal cookies)
- [] Exfiltrate sensitive information
- [] Redirect to phishing site
- [] Perform malicious actions on behalf of users

5.3 DOM-Based XSS

Detection

- [] Analyze client-side JavaScript for unsafe DOM operations:
 - `innerHTML`, `outerHTML`, `document.write()`
 - `eval()`, `Function()`, `setTimeout()`, `setInterval()`
 - `jQuery.html()`, `jQuery.append()`
- [] Trace data flow from source to sink
- [] Test with payloads targeting:
 - `window.location`, URL fragments
 - `document.referrer`
 - `localStorage`, `sessionStorage`

Source & Sink Mapping

- [] Identify all sources (user input points)
 - [] Map to sinks (dangerous functions)
 - [] Test unsanitized data flow
 - [] Verify sanitization effectiveness
-

Phase 6: Cross-Site Request Forgery (CSRF)

6.1 CSRF Detection & Exploitation

Token Validation

- ☐ Check if CSRF tokens are present
- ☐ Verify token uniqueness per session/request
- ☐ Test token reusability across sessions
- ☐ Check if token is tied to user identity
- ☐ Test with null/empty token
- ☐ Test with token from different user's session

Bypass Techniques

- ☐ Test requests without token
- ☐ Test with duplicate token parameter
- ☐ Test with token in different parameter name
- ☐ Test GET vs POST (check if GET is allowed)
- ☐ Test JSON vs form-encoded requests
- ☐ Check if referer header validation is enforced
- ☐ Test origin header manipulation

CSRF Exploitation

- ☐ Create malicious forms for sensitive actions:
 - Password change
 - Email change
 - Settings modification
 - Fund transfer
- ☐ Test CSRF via:
 - HTML forms (auto-submit)
 - Image tags with GET requests
 - AJAX requests with CORS bypass
 - WebSocket connections

6.2 SameSite Cookie Bypass

- ☐ Check SameSite attribute (Strict, Lax, None)
- ☐ Test CSRF with SameSite=Lax bypass (top-level navigation)
- ☐ Verify Secure flag on cookies
- ☐ Test for cookie tossing attacks

Phase 7: Security Misconfigurations

7.1 Server & Application Config

HTTP Security Headers

- ☐ Check for missing/weak:
 - X-Frame-Options (clickjacking protection)
 - X-Content-Type-Options (MIME sniffing)
 - Content-Security-Policy (XSS mitigation)
 - Strict-Transport-Security (HSTS) X-XSS-Protection (legacy XSS filter) Referrer-Policy
 - Permitted-Domains
- ☐ Test for bypassable CSP policies
- ☐ Verify HSTS preload status

Default Credentials & Configuration

- ☐ Check for default admin credentials
- ☐ Look for debug mode enabled in production
- ☐ Check for verbose error messages exposing internals
- ☐ Verify database default passwords
- ☐ Test for exposed management interfaces

Directory & File Exposure

- ☐ Check for directory listing enabled
- ☐ Look for backup files: .bak, .old, .~
- ☐ Find config files: .xml, .conf, .config
- ☐ Look for source code files: .php.bak, .jar~
- ☐ Check .git, .svn, .env exposure
- ☐ Look for log files with sensitive data

Cloud Storage Misconfiguration

- ☐ Enumerate S3 buckets (AWS)
 - Unauthenticated read/write access
 - Public ACLs
 - Bucket policies allowing anonymous access
- ☐ Check Google Cloud Storage (GCS) buckets
- ☐ Enumerate Azure Blob Storage
- ☐ Test for missing authentication on API endpoints

7.2 API Misconfigurations

API Authentication & Authorization

- ☐ Verify all API endpoints require authentication
- ☐ Check for API keys in responses
- ☐ Test for API key reuse across environments
- ☐ Verify authorization on all API methods
- ☐ Check for privilege escalation via API

API Rate Limiting & Throttling

- ☐ Test for missing rate limiting
 - ☐ Check if rate limits can be bypassed:
 - Different user agents
 - IP spoofing
 - X-Forwarded-For header
 - API key rotation
 - ☐ Attempt API abuse (scraping, enumeration)
-

Phase 8: Sensitive Data Exposure

8.1 Data In Transit

HTTPS & TLS

- ☐ Check SSL/TLS certificate validity ☐ Verify proper
- HTTPS implementation on all endpoints ☐ Test for mixed
- content (HTTP + HTTPS) ☐ Check TLS version (min TLS
- 1.2) ☐ Verify cipher suite strength (no weak ciphers) ☐
- Check for HSTS enforcement ☐ Test for SSL stripping
- attacks
-

HTTP Parameter Pollution

- ☐ Test for data leakage in HTTP headers
- ☐ Check for sensitive data in URL parameters
- ☐ Verify data is POST, not GET
- ☐ Check for data in referer headers

8.2 Data At Rest

Sensitive Data Exposure

- ☐ Identify sensitive data storage locations
- ☐ Check for unencrypted sensitive data:
 - Database fields
 - Cache storage
 - Log files
 - Backups
- ☐ Verify encryption key management
- ☐ Test for encryption key hardcoding
- ☐ Check for weak encryption algorithms

Information Disclosure

- ☐ Search for API keys in code/repositories
- ☐ Look for credentials in comments
- ☐ Check for internal IPs/hostnames disclosure
- ☐ Verify stack traces don't expose internals
- ☐ Check for metadata exposure (EXIF, etc.)
- ☐ Look for source code disclosure

- ☐ Test for version information disclosure

8.3 Caching Issues

- ☐ Check cache headers (Cache-Control, Pragma)
- ☐ Test for cached sensitive data
- ☐ Verify cache-busting mechanisms
- ☐ Check for cached authentication responses

Phase 9: File Upload Vulnerabilities

9.1 File Type Bypass

Upload Validation Bypass

- ☐ Test MIME type validation:
 - Change Content-Type header
 - Upload with wrong extension
 - Double extension: shell.php.jpg
 - Null byte: shell.php%00.jpg
 - Case variation: shell.pHp
- ☐ Test magic bytes/ !e signature:
 - Prepend valid !e signature (JPEG header for PHP)
 - Use polyglot !es
- ☐ Test whitelist bypass if applicable

9.2 Path Traversal via Upload

- ☐ Test directory traversal in !lename:
 - ../../../shell.php
 - ..\\..\\..\\shell.php (Windows)
 -/../shell.php (!ter bypass)
- ☐ Check upload destination
- ☐ Verify !e is accessible via web

9.3 Execution of Uploaded Files

Code Execution

- ☐ Upload web shells (.php, .jsp, .asp, .py)
- ☐ Check if uploaded !es are executable
- ☐ Test for Server-Side Template Injection in uploads
- ☐ Look for XXE in XML uploads
- ☐ Check for deserialization vulnerabilities

Other File Upload Attacks

- ☐ Upload !es with EXIF metadata containing payloads
- ☐ SVG !es with embedded JavaScript
- ☐ HTML !es with JavaScript (hosting perspective)
- ☐ PDF !es with embedded scripts
- ☐ O ce documents with macros

9.4 File Inclusion

Local File Inclusion (LFI)

- [] Identify file inclusion parameters
- [] Test basic payloads: `../../../../etc/passwd`
- [] Filter bypass techniques:
 - Encoding: URL, double URL, UTF-8
 - Null bytes: `le.php%00`
 - Traversal bypass: `....//....//`
- [] Test for wrapper usage:
 - `php://filter/convert.base64-encode/resource=index.php`
 - `php://input` for code execution
 - `expect://` for command execution
- [] Verify access to sensitive files

Remote File Inclusion (RFI)

- [] Check if remote files can be included
- [] Test with external payload URLs
- [] Verify protocol restrictions (http, https, ftp)
- [] Test for SSRF via RFI

Phase 10: Cryptographic Failures

10.1 Weak Cryptography

Encryption

- [] Identify encryption usage in application
- [] Check for weak algorithms: DES, RC4, MD5
- [] Verify AES is used with proper mode (CBC, GCM, not ECB)
- [] Check for hardcoded encryption keys
- [] Verify key derivation functions (PBKDF2, bcrypt, scrypt)
- [] Check for proper IV generation (random, not static)

Hashing

- [] Verify password hashing: bcrypt, scrypt, PBKDF2, Argon2
- [] Check for salted hashes
- [] Test for hash reuse attacks
- [] Look for MD5, SHA1 usage (weak for passwords)

10.2 Insecure Random Number Generation

- [] Identify random number usage
- [] Check for weak RNG (`Math.random()`, `rand()`)
- [] Test for predictable random values
- [] Verify proper randomness for:
 - Session tokens
 - CSRF tokens
 - Password reset tokens
 - Encryption keys

10.3 JWT Vulnerabilities

JWT Validation

- [] Verify JWT signature is validated
- [] Check algorithm field (alg) exploitation:
 - Changing alg to "none"
 - Algorithm confusion (HS256 vs RS256)
- [] Test for expired token validation
- [] Check token time claims (iat, exp, nbf)
- [] Verify audience (aud) and issuer (iss) claims

JWT Exploitation

- [] Decode JWT to inspect claims
- [] Test claim manipulation
- [] Look for hardcoded signing keys
- [] Check for weak signing keys (short secrets)
- [] Test kid (key ID) injection

Phase 11: Business Logic Flaws

11.1 Workflow & Process Bypass

Sequential Execution Bypass

- [] Identify multi-step processes (checkout, approval, etc.)
- [] Test skipping steps
- [] Test re-ordering steps
- [] Test revisiting completed steps with modified data
- [] Check for state machine bypass

Price & Discount Manipulation

- [] Test price modification in cart
- [] Check for discount code reuse
- [] Test for negative amounts
- [] Verify discount stacking prevention
- [] Check for currency conversion bugs

Quantity & Inventory Issues

- [] Test negative quantity in orders
- [] Check for inventory decrement bypass
- [] Test concurrent purchase of limited items
- [] Verify stock quantity validation

11.2 Race Conditions

Concurrent Request Handling

- [] Test simultaneous requests for:
 - Fund transfers
 - Inventory purchases
 - Coupon redemption
 - Email/password change
- [] Use threading/parallel requests
- [] Check for time-of-check-time-of-use (TOCTOU) bugs

Account-Related Race Conditions

- [] Test concurrent password reset requests
- [] Test simultaneous account deletion
- [] Check for double billing in concurrent charges

11.3 Broken Business Logic

Authorization & Privilege

- [] Test unauthorized role assignment
- [] Check for privilege escalation via business logic
- [] Verify action authorization matches displayed permissions
- [] Test for privilege downgrade prevention

Data Consistency

- [] Test for duplicate transactions
- [] Check for orphaned data
- [] Verify referential integrity
- [] Test for data desynchronization

Phase 12: Server-Side Request Forgery (SSRF)

12.1 SSRF Detection & Exploitation

Identification

- [] Identify functions that make HTTP requests:
 - Image proxy endpoints
 - Webhook functionality
 - URL shorteners
 - PDF generators
 - API integrations
- [] Test with external URL
- [] Check for DNS resolution

Exploitation

- ☐ Test access to internal services:
 - `http://localhost:8080`
 - `http://127.0.0.1:8080`
 - `http://192.168.1.1` (local network)
 - `http://169.254.169.254` (AWS metadata)
- ☐ Test cloud metadata services:
 - AWS: `http://169.254.169.254/latest/meta-data/`
 - GCP: `http://metadata.google.internal/`
 - Azure: `http://169.254.169.254/metadata/instance/`

Filter Bypass

- ☐ Test encoding: URL encoding, unicode, punycode
 - ☐ Case variation: `Http://localhost`
 - ☐ IP bypasses:
 - Hex encoding: `http://0x7f000001`
 - Octal: `http://0177.0.0.1`
 - Decimal: `http://2130706433`
 - Domain: `localhost.localhost`, `127.0.0.1.nip.io`
 - ☐ Test with @ character: `http://attacker.com@localhost`
 - ☐ Test URL parser bypass
-

Phase 13: Deserialization Vulnerabilities

13.1 Unsafe Deserialization

Detection

- ☐ Identify deserialization in:
 - Session storage
 - Cookie values
 - API responses
 - File uploads
- ☐ Check serialization format: Java, PHP, Python, .NET, Ruby

Exploitation

- ☐ Java: Use ysoserial for gadget chain generation
 - ☐ PHP: Test for object injection
 - ☐ Python: Pickle/PyYAML deserialization
 - ☐ .NET: BinaryFormatter vulnerabilities
 - ☐ Look for magic methods (`__wakeup`, `__destruct`)
-

Phase 14: Component Vulnerabilities

14.1 Known Vulnerabilities

Dependency Scanning

- ☐ Identify all third-party libraries:
 - Server-side (requirements.txt, package.json, pom.xml)
 - Client-side (npm, bower, CDN)
- ☐ Check for known CVEs:
 - Use tools: npm audit, safety, dependency-check
 - Search: CVE databases, NIST, Exploit-DB
- ☐ Verify component versions
- ☐ Check for vulnerable dependencies of dependencies

Exploitation

- ☐ Research available exploits for vulnerable components
- ☐ Test for component exploitation
- ☐ Chain vulnerabilities for greater impact

14.2 Unpatched Systems

- ☐ Check server OS for security patches
- ☐ Verify application framework patching
- ☐ Look for outdated dependencies

Phase 15: API-Specific Vulnerabilities

15.1 GraphQL Vulnerabilities

Query Introspection

- ☐ Check if introspection is enabled
- ☐ Enumerate all available types, queries, mutations
- ☐ Identify sensitive fields and operations

GraphQL Injection & Exploitation

- ☐ Test for GraphQL injection in query fields
- ☐ Attempt alias-based DoS
- ☐ Test nested query depth limits
- ☐ Look for circular fragment exploitation
- ☐ Check rate limiting on GraphQL operations

Authorization in GraphQL

- ☐ Test field-level authorization bypass
- ☐ Check mutation authorization
- ☐ Verify user context is enforced

15.2 REST API Vulnerabilities

API Enumeration

- ☐ Discover all API endpoints ☐
- Identify HTTP methods per endpoint ☐
- Document request/response formats ☐
- Check API versioning

API Authentication & Authorization

- ☐ Verify authentication on all endpoints
- ☐ Test authorization per HTTP method
- ☐ Check for privilege escalation via API

API Rate Limiting

- ☐ Test for missing rate limits
- ☐ Attempt API abuse (scraping, enumeration)
- ☐ Check rate limit bypass methods

15.3 SOAP & XML Services

- ☐ Test for XXE in SOAP requests
- ☐ Check for XML bombing (Billion Laughs attack)
- ☐ Verify WSDL exposure
- ☐ Test for SOAP injection

Phase 16: Subdomain Takeover

16.1 Detection & Exploitation

Identification

- ☐ Enumerate all subdomains
- ☐ Check DNS resolution
- ☐ Identify dangling DNS records
- ☐ Look for subdomains pointing to removed services

Common Takeover Targets

- ☐ GitHub Pages: `your-username.github.io`
- ☐ Heroku: `app-name.herokuapp.com`
- ☐ Amazon S3: `bucket.s3.amazonaws.com`
- ☐ Google Cloud Storage
- ☐ Azure services
- ☐ Expired domains

Exploitation

- [] Claim the service (GitHub repo, Heroku app, etc.)
- [] Host malicious content
- [] Test for XSS/phishing via subdomain

Phase 17: Advanced Techniques & Automation

17.1 Reconnaissance Automation

Subdomain enumeration

```
subfinder -d target.com -o subs.txt  
cat subs.txt | httpprobe > alive_subs.txt
```

URL discovery

```
cat alive_subs.txt | waybackurls | sort -u > urls.txt  
cat alive_subs.txt | while read sub; do gau $sub >> urls.txt; done
```

Sensitive endpoint finding

```
cat urls.txt | grep -E "(admin|api|config|backup|test|debug)"
```

Technology identification

```
cat alive_subs.txt | while read sub; do curl -s -I $sub | grep -E "(Server|X-Powered-By|X-AspNet-Version)"; done
```

17.2 Vulnerability Scanning Automation

Nuclei scanning for templates

```
nuclei -l alive_subs.txt -o nuclei_results.txt
```

OWASP ZAP automated scan

```
zapproxy -cmd -quickurl http://target.com -quickout zap_report.html
```

Burp Suite - Automated scanning via command line

(Requires Burp Suite Professional)

17.3 Manual Testing Priorities

- ☐ High-value endpoints (login, admin, API)
- ☐ User input validation (search, filter, profile)
- ☐ Complex workflows (checkout, approvals)
- ☐ File operations (upload, download, preview)
- ☐ Data retrieval and enumeration

Phase 18: Exploitation & Proof of Concept

18.1 PoC Development

Goals

- ☐ Demonstrate vulnerability clearly
- ☐ Minimize environmental assumptions
- ☐ Create reproducible steps
- ☐ Document prerequisites
- ☐ Provide remediation guidance

Documentation

- ☐ Vulnerability title and CVE (if applicable)
- ☐ CVSS score estimation
- ☐ Affected versions/configurations
- ☐ Step-by-step reproduction steps
- ☐ Expected vs actual results
- ☐ Impact statement
- ☐ Remediation recommendations

Code & Media

- ☐ Attach exploit code/scripts
- ☐ Include screenshots/video if applicable
- ☐ Provide sample requests (curl, raw HTTP)
- ☐ Include response data for reference

18.2 Impact Assessment

- ☐ Confidentiality impact (data exposure)
 - ☐ Integrity impact (data modification)
 - ☐ Availability impact (service disruption)
 - ☐ Business impact (financial, reputation, legal)
 - ☐ User impact (number of affected users)
-

Phase 19: Reporting & Disclosure

19.1 Vulnerability Report Structure

1. Executive Summary
 - Vulnerability title
 - Severity rating
 - Brief impact description
2. Vulnerability Details
 - Description
 - Type (OWASP Top 10 category)
 - Affected component/endpoint
 - Prerequisites (if any)
3. Proof of Concept
 - Step-by-step reproduction
 - Screenshots/video
 - Sample payloads
 - Expected behavior
4. Impact
 - Technical impact
 - Business impact
 - Risk assessment
5. Remediation
 - Fix recommendations
 - Code examples (if applicable)
 - Alternative mitigations
 - Timeline for remediation
6. References
 - OWASP resources
 - CWE/CVE links
 - External references

19.2 Disclosure Best Practices

- ☐ Follow responsible disclosure timeline
- ☐ Avoid public disclosure before patch
- ☐ Provide sufficient time for remediation (30-90 days typical)
- ☐ Escalate if no response after timeline
- ☐ Document all communication
- ☐ Coordinate multi-vendor disclosure if needed

Phase 20: Continuous Monitoring & Retesting

20.1 Retesting After Fix

- ☐ Verify all vulnerability instances are fixed
- ☐ Check for incomplete patches
- ☐ Test for similar vulnerabilities in same codebase
- ☐ Verify no regression (new vulnerabilities introduced)

20.2 Program Monitoring

- [] Monitor program announcements
- [] Track scope changes
- [] Check for new subdomains/assets
- [] Monitor for reappearing vulnerabilities
- [] Engage with program community

Appendix: Tool Reference

Reconnaissance Tools

- Sub nder, asset nder, amass, dnsx
- Waybackurls, gau, hakrawler
- Wappalyzer, BuiltWith, Shodan
- crt.sh, SSL Labs

Scanning & Enumeration

Nmap, masscan

Nuclei, Burp Suite, OWASP ZAP

Nikto, dirbuster

Exploitation Tools

- Burp Suite
- SQLmap, NoSQL injection tools
- XXE payloads, SSTI templates
- JWT tools (jwt.io, jwtcat)
- Ysoserial (Java deserialization)

Utility Tools

- Curl, Postman, Insomnia
- Python scripting (requests, urllib3)
- Bash scripting
- Grep, sed, awk for parsing

Reporting

- Jira, GitHub Issues
 - Markdown, LaTeX for documentation
 - Screenshot tools (Flameshot, Snagit)
 - Video recording (OBS, Loom)
-

